Administration & Tuning des Bases de Données

Guide de révision complet – M1 SSI – USTHB

Dr. Ferrahi Ibtisam

Ce document contient :

- Analyse des examens 2017, 2019, 2020, 2021 – questions répétées identifiées
- Résumé complet des 4 chapitres de cours
- Résumé des 3 TPs
- Tout le SQL et PL/SQL expliqués de zéro

2025/2026

Table des matières

1 Questions répétées aux examens (priorité absolue) 2
- 1.1 Question 1 – Création complète d'une table avec contraintes . . . 2
- 1.2 Question 2 – Ajouter des contraintes à une table existante (ALTER TABLE) . . . 3
- 1.3 Question 3 – Trigger pour contrainte dynamique . . . 3
- 1.4 Question 4 – Gestion des utilisateurs, tablespaces, rôles, profils . . . 5
- 1.5 Question 5 – Cas d'échec d'une requête INSERT . . . 6
- 1.6 Question 6 – Plan d'exécution (EXPLAIN PLAN) . . . 6
- 1.7 Question 7 – Arbre algébrique + optimisation RBO . . . 7
- 1.8 Question 8 – Index Bitmap . . . 7
- 1.9 Question 9 – Fragmentation horizontale (PARTITION) . . . 8

2 Chapitre 1 – Introduction : Architecture des SGBD 10
- 2.1 Définitions fondamentales . . . 10
- 2.2 Architecture à 3 niveaux (ANSI/SPARC) . . . 10
- 2.3 Composants fonctionnels d'un SGBD . . . 10

3 Chapitre 2 – Sécurité, Utilisateurs, Tablespaces, Privilèges, Rôles 11
- 3.1 Structure logique et physique . . . 11
- 3.2 Gestion des utilisateurs . . . 11
- 3.3 Gestion des profils . . . 12
- 3.4 Gestion des privilèges . . . 12
- 3.5 Gestion des rôles . . . 13
- 3.6 Dictionnaire de données (TP3) . . . 14

4 Chapitre 3 – Contraintes d'intégrité et PL/SQL 16
- 4.1 Types de contraintes . . . 16
- 4.2 PL/SQL – Tout ce qu'il faut savoir . . . 16
  - 4.2.1 Structure d'un bloc PL/SQL . . . 16
  - 4.2.2 Instructions de contrôle . . . 17
  - 4.2.3 SELECT INTO (mono-ligne) . . . 17
  - 4.2.4 Curseurs (sélection multi-lignes) . . . 17
  - 4.2.5 Procédures et fonctions stockées . . . 18
  - 4.2.6 Gestion des exceptions . . . 19

5 Chapitre 4 – Optimisation des requêtes 20
- 5.1 Les deux approches . . . 20
- 5.2 Propriétés algébriques (pour construire l'arbre optimisé) . . . 20
- 5.3 Algorithmes de jointure . . . 20
- 5.4 Hints Oracle . . . 21

6 SQL de A à Z – Tout ce qu'il faut maîtriser 22
- 6.1 DDL – Définition des données . . . 22
- 6.2 DML – Manipulation des données . . . 22
- 6.3 DQL – Interrogation . . . 23
- 6.4 Fonctions SQL utiles . . . 24
- 6.5 Sous-requêtes et opérateurs ensemblistes . . . 24

7 Résumé des TPs **25**
7.1 TP1 – Création et manipulation de BD 25
7.2 TP2 – Gestion des droits 25
7.3 TP3 – Dictionnaire de données 26

8 Index B-arbre (question 8 des examens 2017 et docx) **28**

9 SQL*Loader (chargement de données depuis fichier) **29**

10 Fiche de révision ultra-rapide (dernière heure avant l'exam) **30**

1 Questions répétées aux examens (priorité absolue)

Question répétée aux examens

Analyse des examens 2017, 2019, 2020 et du sujet docx (même BD assurance) : les mêmes 9 types de questions reviennent à **chaque session**. Maîtriser ces 9 blocs = passer l'examen.

1.1 Question 1 – Création complète d'une table avec contraintes

Présente dans TOUS les examens. Toujours une table avec PK + 2 FK + éventuellement CHECK.

Question répétée aux examens

Exemple type (2017 et docx, même sujet) : Créer la table CONTRAT(NOCONTRAT, NOASSURE*, NOVEHICULE*, DATE-C, NATURE, PRIME).

Listing 1 – Patron universel de création de table

```sql
CREATE TABLE Contrat (
  NOCONTRAT    NUMBER,
  NOASSURE     NUMBER        NOT NULL,
  NOVEHICULE   NUMBER        NOT NULL,
  DATE_C       DATE,
  NATURE       VARCHAR2(30),
  PRIME        NUMBER(10,2),
  -- Contrainte PK
  CONSTRAINT pk_contrat PRIMARY KEY (NOCONTRAT),
  -- Contraintes FK
  CONSTRAINT fk_ct_ass FOREIGN KEY (NOASSURE)
    REFERENCES ASSURE(NOASSURE),
  CONSTRAINT fk_ct_veh FOREIGN KEY (NOVEHICULE)
    REFERENCES VEHICULE(NOVEHICULE)
);
```

Attention / Piège fréquent

Erreurs classiques à ne PAS faire :

- Oublier le mot-clé `CONSTRAINT nom_contrainte` avant chaque contrainte
- Écrire `FOREIGN KEY` sans le nom de la contrainte
- Référencer une table parente qui n'existe pas encore (créer dans l'ordre !)

Explication de chaque type de contrainte :

- `PRIMARY KEY` : identifiant unique de la ligne, interdit NULL, crée automatiquement un index unique.

— `FOREIGN KEY ... REFERENCES` : garantit qu'une valeur existe bien dans la table parente. Options : `ON DELETE CASCADE` (suppression en cascade), `ON DELETE SET NULL` (met NULL), pas d'option = rejet de la suppression du parent.

— `NOT NULL` : la colonne ne peut pas rester vide.

— `UNIQUE` : toutes les valeurs doivent être différentes (NULL autorisé).

— `CHECK(condition)` : impose une règle logique. Ex : `CHECK(AGE BETWEEN 18 AND 60)`.

1.2 Question 2 – Ajouter des contraintes à une table existante (ALTER TABLE)

Présente dans l'exam 2019 (toute la BD créée sans contraintes, puis les ajouter).

Listing 2 – Ajouter contraintes après coup

```sql
-- Cl   primaire
ALTER TABLE Societe ADD CONSTRAINT pk_soc PRIMARY KEY (NomSociete);

-- Cl      trangre
ALTER TABLE Filiale ADD CONSTRAINT fk_fil_soc
  FOREIGN KEY (NomSociete) REFERENCES Societe(NomSociete);

-- Contrainte CHECK
ALTER TABLE Employe ADD CONSTRAINT chk_grade
  CHECK (Grade IN ('TS','Ingenieur','Ingenieur Principal','
     Administrateur','Gerant'));

-- Contrainte CHECK sur plage
ALTER TABLE Employe ADD CONSTRAINT chk_age CHECK (Age BETWEEN 18 AND
   60);

-- Contrainte multi-colonnes (ex: date fin > date debut)
ALTER TABLE Produit ADD CONSTRAINT chk_dates
  CHECK (Date_Fin_Garantie >= Date_Fab + INTERVAL '1' YEAR);
```

1.3 Question 3 – Trigger pour contrainte dynamique

Présente dans TOUS les examens (2017, 2019, 2020, docx). Toujours : vérifier une contrainte temporelle ou un calcul à l'insertion/mise à jour.

Question répétée aux examens

Variantes vues aux examens :

— **2017 & docx (même question) :** La date du sinistre doit être > date du contrat.

— **2019 :** Mettre à jour automatiquement le montant global d'une commande.

— **2020 :** Empêcher une date de création d'équipe d'être antérieure à l'ancienne valeur.

— **2020 :** Calculer le temps global par équipe/étape.

Listing 3 – Trigger vérification de date (2017, docx – IDENTIQUE)

```oracle_plsql
CREATE OR REPLACE TRIGGER verif_date
BEFORE INSERT ON Sinistre
FOR EACH ROW
DECLARE
  d Contrat.DATE_C%TYPE;
BEGIN
  SELECT DATE_C INTO d
  FROM Contrat
  WHERE NOCONTRAT = :NEW.NOCONTRAT;

  IF :NEW.DATE_S < d THEN
    RAISE_APPLICATION_ERROR(-20001,
      'La date du sinistre doit etre superieure a la date du contrat');
  END IF;
END;
/
```

Listing 4 – Trigger mise à jour automatique d'un montant (2019)

```oracle_plsql
CREATE OR REPLACE TRIGGER maj_montant_cmd
AFTER INSERT ON Ligne_Commande
FOR EACH ROW
DECLARE
  prix Produit.Prix_Unitaire%TYPE;
BEGIN
  SELECT Prix_Unitaire INTO prix
  FROM Produit WHERE NumProd = :NEW.NumProd;

  UPDATE Commande
  SET MontantG = MontantG + (:NEW.NB_Unite * prix)
  WHERE NumCMD = :NEW.NumCMD;
END;
/
```

Listing 5 – Trigger anti-régression de date (2020)

```oracle_plsql
CREATE OR REPLACE TRIGGER verif_date_equipe
BEFORE UPDATE ON Equipe
FOR EACH ROW
BEGIN
  IF :NEW.Date_Creation < :OLD.Date_Creation THEN
    RAISE_APPLICATION_ERROR(-20002,
      'Impossible de reculer la date de creation');
  END IF;
END;
/
```

Anatomie d'un trigger PL/SQL :

- `BEFORE / AFTER` : s'exécute avant ou après l'événement DML.
- `INSERT / UPDATE / DELETE` : l'événement déclencheur.

— `FOR EACH ROW` : trigger "ligne" (déclenché pour chaque ligne affectée). Sans = trigger "global" (une seule fois).

— `:NEW.colonne` : nouvelle valeur (INSERT, UPDATE). `:OLD.colonne` : ancienne valeur (UPDATE, DELETE).

— `RAISE_APPLICATION_ERROR(code, message)` : génère une erreur. Le code doit être entre -20000 et -20999.

1.4 Question 4 – Gestion des utilisateurs, tablespaces, rôles, profils

Présente dans les examens 2019, 2020. Toujours : créer tablespace + user + rôle + profil + attribuer.

Listing 6 – Chaîne complète : tablespace → user → rôle → profil (2019)

```
-- 1. Cr er les tablespaces
CREATE TABLESPACE INFO_TBS
  DATAFILE 'tbs_info.dat' SIZE 15000M AUTOEXTEND ON;

CREATE TEMPORARY TABLESPACE INFO_TEMP_TBS
  TEMPFILE 'tbs_temp_info.dat' SIZE 500M AUTOEXTEND ON;

-- 2. Cr er l'utilisateur
CREATE USER INFO_COM IDENTIFIED BY monpass
  DEFAULT TABLESPACE INFO_TBS   QUOTA 7500M ON INFO_TBS
  TEMPORARY TABLESPACE INFO_TEMP_TBS;

-- 3. Cr er un r le et lui donner des privil ges
CREATE ROLE INFCMD;
GRANT INSERT, DELETE, UPDATE ON Commande       TO INFCMD;
GRANT INSERT, DELETE, UPDATE ON Ligne_Commande TO INFCMD;
GRANT SELECT ON Produit     TO INFCMD;
GRANT SELECT ON Fournisseur TO INFCMD;
GRANT CREATE ANY INDEX      TO INFCMD;

-- Privil ge PUBLIC (tous les utilisateurs)
GRANT SELECT ON Filiale TO PUBLIC;

-- 4. Attribuer le r le    l'utilisateur
GRANT INFCMD TO INFO_COM;
```

Listing 7 – Profil avec limites de connexion (2020)

```
CREATE PROFILE INVITE_EXTERNE_PR LIMIT
  SESSIONS_PER_USER       1
  CONNECT_TIME            30
  FAILED_LOGIN_ATTEMPTS   4
  PASSWORD_LOCK_TIME      7;

-- Cr er l'utilisateur avec profil
CREATE USER Extern_User10 IDENTIFIED BY pass123
  PROFILE INVITE_EXTERNE_PR;

```

```
-- Attribuer le r le
GRANT INVITE_EXTERNE TO Extern_User10;
```

1.5 Question 5 – Cas d'échec d'une requête INSERT

Présente dans 2017 & docx (même question exacte). Toujours : lister les raisons d'échec + solutions.

Question répétée aux examens

Question exacte répétée (2017 = docx) : `Insert into SINISTRE (12, 10, '15/10/2016', 'Chaussée Glissante')` – dans quels cas échoue-t-elle ?

Les raisons d'échec d'un INSERT sont toujours dans ces catégories :

Cause	Erreur Oracle	Solution
Clé primaire dupliquée	ORA-00001	Changer la valeur de la PK
FK inexistante (parent absent)	ORA-02291	Créer d'abord le parent
Violation du trigger (date)	ORA-20xxx	Corriger la date
Pas de droit INSERT	ORA-01031	`GRANT INSERT ON table TO user`
Tablespace saturé	ORA-01653	`ALTER TABLESPACE ... AUTOEXTEND ON`
Violation contrainte CHECK	ORA-02290	Corriger la valeur

1.6 Question 6 – Plan d'exécution (EXPLAIN PLAN)

Présente dans 2017, docx (plan identique !). Lire et expliquer chaque ligne du plan.

Question répétée aux examens

Plan identique dans 2017 et dans le docx : TABLE ACCESS FULL sur Sinistre + INDEX UNIQUE SCAN sur PK_Contrat + TABLE ACCESS BY INDEX ROWID sur Vehicule + HASH JOIN + NESTED LOOPS.

Listing 8 – Afficher le plan d'exécution

```
-- M thode 1 : EXPLAIN PLAN
EXPLAIN PLAN FOR SELECT * FROM Sinistre s, Contrat c WHERE s.NOCONTRAT
    = c.NOCONTRAT;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

-- M thode 2 : autotrace (dans SQL*Plus)
SET AUTOTRACE ON
```

Vocabulaire du plan (à mémoriser) :

- **TABLE ACCESS FULL** : lit toute la table séquentiellement. Coûteux sur grandes tables.
- **INDEX UNIQUE SCAN** : cherche exactement 1 valeur via l'index (clé primaire ou UNIQUE). Très rapide.

- **INDEX RANGE SCAN** : parcourt une plage de valeurs dans l'index (ex : AGE > 30).
- **TABLE ACCESS BY INDEX ROWID** : accède à la table grâce au ROWID récupéré depuis l'index.
- **HASH JOIN** : jointure par hachage. Efficace pour grandes tables.
- **NESTED LOOPS** : jointure par boucles imbriquées. Efficace si une table est petite ou indexée.
- **MERGE JOIN** : jointure par tri-fusion. Efficace si les deux tables sont déjà triées.

1.7 Question 7 – Arbre algébrique + optimisation RBO

Présente dans 2019 et 2020. Toujours : SQL → arbre non optimisé → arbre optimisé (RBO) → calcul de coût.

Principe de l'optimisation RBO (Rule-Based Optimization) :

1. **Descendre les sélections (restrictions) le plus bas possible** dans l'arbre, vers les feuilles (les tables).
2. **Appliquer les projections tôt** pour réduire la taille des tuples.
3. **Remplacer produit cartésien + sélection par jointure** quand la condition porte sur les deux tables.

Formules de facteur de sélectivité (FS) – à mémoriser pour le calcul de coût :

$$\begin{aligned}
\mathrm{FS}(A = \mathrm{val}) &= \frac{1}{\mathrm{NDIST}(A)} \\
\mathrm{FS}(A > \mathrm{val}) &= \frac{\max(A) - \mathrm{val}}{\max(A) - \min(A)} \\
\mathrm{FS}(A < \mathrm{val}) &= \frac{\mathrm{val} - \min(A)}{\max(A) - \min(A)} \\
\mathrm{FS}(P \text{ AND } Q) &= \mathrm{FS}(P) \times \mathrm{FS}(Q) \\
||R_{\text{résultat}}|| &= \mathrm{FS} \times ||R||
\end{aligned}$$

Calcul de coût (CBO) :

- Coût = nombre de pages (blocs) transférées entre disque et RAM.
- Nombre de pages d'une table : $|T| = \lceil ||T|| \times T_tuple/\mathrm{PS} \rceil$
- **Jointure par boucles imbriquées :** Coût $= |R| + |R| \times |S|$ (ou $|R| + |S|$ si R tient en RAM).
- **Jointure par hachage :** Coût $= 3(|R| + |S|)$.
- **Jointure par tri-fusion :** Coût $= 2|R| \log |R| + 2|S| \log |S| + |R| + |S|$ (si non triées).
- Jointure sur clé étrangère : $||R_1 \bowtie R_2|| = ||R_1||$ (chaque tuple de R1 se joint à exactement un de R2).

1.8 Question 8 – Index Bitmap

Présente dans 2019 (index bitmap sur Grade et Age) et 2017 (index bitmap sur Nature du contrat).

Listing 9 – Création d'un index bitmap

```
-- Sur un attribut     faible cardinalit  (peu de valeurs distinctes)
CREATE BITMAP INDEX grade_idx   ON Employe(Grade);
CREATE BITMAP INDEX age_idx     ON Employe(Age);
CREATE BITMAP INDEX nature_idx  ON Contrat(Nature);
```

Taille d'un index bitmap :

$$\text{Taille (bits)} = ||T|| \times \text{NDIST(attribut)}$$

$$\text{Taille (Ko)} = \frac{||T|| \times \text{NDIST}}{8 \times 1024}$$

Utilisation : Oracle charge les colonnes de bits correspondant aux valeurs cherchées, effectue des opérations AND/OR bitwise, récupère les ROWID correspondant à 1, puis charge les tuples.

1.9 Question 9 – Fragmentation horizontale (PARTITION)

Présente dans 2017, 2019 et docx. Toujours : fragmenter une table sur un attribut, puis fragmentation dérivée.

Listing 10 – Fragmentation horizontale par RANGE (2017, docx – similaire)

```
-- Table principale : fragmentation par plage d' ge
CREATE TABLE Assure_Frag (
  NOASSURE   NUMBER PRIMARY KEY,
  NOM_ASS    VARCHAR2(50),
  AGE_ASS    NUMBER,
  WILAYA_ASS VARCHAR2(30)
)
PARTITION BY RANGE (AGE_ASS) (
  PARTITION Ass_moins30 VALUES LESS THAN (31),
  PARTITION Ass_31_60   VALUES LESS THAN (61),
  PARTITION Ass_plus60  VALUES LESS THAN (MAXVALUE)
);

-- Recopier les donn es et renommer
INSERT INTO Assure_Frag SELECT * FROM Assure;
DROP TABLE Assure;
ALTER TABLE Assure_Frag RENAME TO Assure;
```

Listing 11 – Fragmentation dérivée par REFERENCE (table enfant)

```
-- La table Contrat se fragmente automatiquement selon la FK vers
   Assure
CREATE TABLE Contrat_Frag (
  NOCONTRAT  NUMBER,
  NOASSURE   NUMBER,
  DATE_C     DATE,
  NATURE     VARCHAR2(30),
  PRIME      NUMBER(10,2),
  CONSTRAINT pk_contrat PRIMARY KEY (NOCONTRAT),
```

```
  CONSTRAINT fk_ct_ass FOREIGN KEY (NOASSURE) REFERENCES Assure(
      NOASSURE)
)
PARTITION BY REFERENCE (fk_ct_ass);
```

Fragmentation par LIST : quand les valeurs ne sont pas ordonnées.

```
PARTITION BY LIST (Domaine) (
  PARTITION Soc_Smart  VALUES ('Smartphone'),
  PARTITION Soc_Autre  DEFAULT
);
```

2 Chapitre 1 – Introduction : Architecture des SGBD

Résumé clé

Ce chapitre sert d'introduction générale. Il n'a pas généré de questions directes aux examens, mais le vocabulaire est nécessaire pour comprendre les questions d'optimisation.

2.1 Définitions fondamentales

- **Base de données (BD) :** collection de données cohérentes, persistantes et structurées, décrites par un schéma.
- **SGBD :** logiciel gérant la BD. Il permet de consulter, modifier, sauvegarder. Oracle est un SGBDR (Relationnel) + objet depuis la v8.
- **DBA (Data Base Administrator) :** installe le SGBD, gère espaces disques, utilisateurs, sécurité, sauvegardes, performances.

2.2 Architecture à 3 niveaux (ANSI/SPARC)

- **Schéma interne :** organisation physique des données sur disque.
- **Schéma conceptuel :** description logique (tables, contraintes).
- **Schéma externe :** vues partielles pour les applications/utilisateurs.

2.3 Composants fonctionnels d'un SGBD

1. **Interface utilisateur :** reçoit les requêtes SQL.
2. **Analyseur de requêtes :** vérifie la syntaxe, génère l'arbre algébrique.
3. **Optimiseur :** choisit le meilleur plan d'exécution (RBO ou CBO).
4. **Gestionnaire de transactions :** propriétés ACID.
5. **Gestionnaire de verrous (Lock Manager) :** isolation des transactions concurrentes.
6. **Buffer Manager :** gère le cache entre disque et RAM.
7. **Gestionnaire de métadonnées :** dictionnaire de données.

3 Chapitre 2 – Sécurité, Utilisateurs, Tablespaces, Privilèges, Rôles

Résumé clé

Chapitre le plus important pour les examens. Toutes les questions de la Partie I des examens viennent d'ici.

3.1 Structure logique et physique

Tablespace : unité logique de stockage composée d'un ou plusieurs fichiers physiques (.dbf).

- Un tablespace contient des **segments** (tables, index...).
- Un segment contient des **extents** (ensembles de blocs contigus).
- Un extent contient des **blocs Oracle** (unité minimale).

Listing 12 – Créer un tablespace permanent

```
CREATE TABLESPACE mon_tbs
  DATAFILE 'C:\oracle\mon_tbs.dbf' SIZE 100M
  AUTOEXTEND ON NEXT 10M MAXSIZE 500M
  EXTENT MANAGEMENT LOCAL AUTOALLOCATE
  SEGMENT SPACE MANAGEMENT AUTO
  ONLINE;
```

Listing 13 – Créer un tablespace temporaire

```
CREATE TEMPORARY TABLESPACE mon_temp
  TEMPFILE 'C:\oracle\mon_temp.dbf' SIZE 50M AUTOEXTEND ON;
```

Opérations sur tablespace :

```
-- Mettre offline/online
ALTER TABLESPACE mon_tbs OFFLINE;
ALTER TABLESPACE mon_tbs ONLINE;

-- Ajouter un fichier de donn es
ALTER TABLESPACE mon_tbs ADD DATAFILE 'fichier2.dbf' SIZE 50M;

-- Supprimer
DROP TABLESPACE mon_tbs INCLUDING CONTENTS AND DATAFILES CASCADE
   CONSTRAINTS;
```

3.2 Gestion des utilisateurs

Listing 14 – Créer un utilisateur (syntaxe complète)

```
CREATE USER nom_user
  IDENTIFIED BY mot_de_passe
  DEFAULT TABLESPACE mon_tbs        -- tablespace par d faut
  QUOTA 10M ON mon_tbs              -- espace max autoris
```

```
  TEMPORARY TABLESPACE mon_temp
  PASSWORD EXPIRE                    -- oblige    changer le mdp    la 1
      re  connexion
  ACCOUNT UNLOCK                     -- compte d verrouill  par d faut
  PROFILE mon_profil;                -- profil de ressources
```

Listing 15 – Modifier et supprimer un utilisateur

```
-- Modifier un quota
ALTER USER nom_user QUOTA UNLIMITED ON mon_tbs;

-- Verrouiller/d verrouiller un compte
ALTER USER nom_user ACCOUNT LOCK;
ALTER USER nom_user ACCOUNT UNLOCK;

-- Changer le mot de passe
ALTER USER nom_user IDENTIFIED BY nouveau_mdp;

-- Supprimer (CASCADE supprime tous ses objets)
DROP USER nom_user CASCADE;
```

3.3 Gestion des profils

Un profil regroupe des limites sur les ressources système et les mots de passe.

Listing 16 – Créer un profil

```
CREATE PROFILE mon_profil LIMIT
  -- Ressources syst me
  SESSIONS_PER_USER   2        -- max 2 sessions simultan es
  CPU_PER_SESSION     10000    -- en centi mes de secondes
  CONNECT_TIME        480      -- 8h max par session
  IDLE_TIME           60       -- 1h d'inactivit  max
  -- Gestion des mots de passe
  FAILED_LOGIN_ATTEMPTS  3     -- blocage apr s 3 tentatives
  PASSWORD_LOCK_TIME     1     -- bloqu  1 jour
  PASSWORD_LIFE_TIME     30    -- expire apr s 30 jours
  PASSWORD_GRACE_TIME    5     -- 5 jours de gr ce
  PASSWORD_REUSE_TIME    180;  -- ne peut r utiliser avant 180 jours
```

```
-- Activer les limites de ressources (n cessaire !)
ALTER SYSTEM SET RESOURCE_LIMIT = TRUE;

-- Affecter un profil
ALTER USER nom_user PROFILE mon_profil;

-- Supprimer un profil (impossible pour DEFAULT)
DROP PROFILE mon_profil;
```

3.4 Gestion des privilèges

Deux catégories :

— **Privilèges système :** opérations globales sur la BD. Ex : `CREATE TABLE`, `CREATE USER`, `CREATE ANY INDEX`.

— **Privilèges objet :** accès à un objet spécifique. Ex : `SELECT ON table`, `UPDATE ON table`.

Listing 17 – GRANT et REVOKE

```sql
-- Privil ge syst me
GRANT CREATE TABLE TO nom_user;
GRANT CREATE TABLE TO nom_user WITH ADMIN OPTION; -- il peut re-donner
    ce droit

-- Privil ge objet
GRANT SELECT, INSERT ON ma_table TO nom_user;
GRANT UPDATE (col1, col2) ON ma_table TO nom_user; -- colonnes
    sp cifiques
GRANT ALL PRIVILEGES ON ma_table TO nom_user;
GRANT SELECT ON ma_table TO PUBLIC; -- tous les utilisateurs
GRANT SELECT ON ma_table TO nom_user WITH GRANT OPTION; -- peut re-
    donner

-- R voquer
REVOKE CREATE TABLE FROM nom_user;
REVOKE SELECT ON ma_table FROM nom_user;
REVOKE ALL PRIVILEGES ON ma_table FROM nom_user;
REVOKE SELECT ON ma_table FROM PUBLIC;
```

Attention / Piège fréquent

RESTRICT vs CASCADE lors d'un REVOKE : Si A donne le privilège P à B, et B le donne à C, quand A révoque P à B :

— **RESTRICT :** C garde P (si P lui a été donné par quelqu'un d'autre aussi).

— **CASCADE :** C perd P sauf si quelqu'un d'autre lui a aussi accordé P.

Privilège objet	Table	Vue	Séquence	Procédure
SELECT	✓	✓	✓	
INSERT	✓	✓		
UPDATE	✓	✓		
DELETE	✓	✓		
ALTER	✓		✓	
INDEX	✓			
EXECUTE				✓
REFERENCES	✓	✓	✓	

3.5 Gestion des rôles

Un rôle = groupe de privilèges réutilisable.

```sql
-- Cr er un r le
CREATE ROLE mon_role;
CREATE ROLE mon_role IDENTIFIED BY mdp_role; -- avec mot de passe

-- Donner des privil ges au r le
GRANT SELECT ON table1 TO mon_role;
GRANT INSERT ON table2 TO mon_role;

-- Donner le r le    un utilisateur
GRANT mon_role TO nom_user;

-- Supprimer un r le
DROP ROLE mon_role;

-- Activer/d sactiver un r le dans la session
SET ROLE mon_role;
SET ROLE ALL;
SET ROLE NONE;
```

Rôles prédéfinis Oracle : CONNECT (connexion basique), RESOURCE (créer tables/index), DBA (tous les privilèges avec ADMIN OPTION).

3.6 Dictionnaire de données (TP3)

Vue	Contenu
USER_TABLES	Mes tables
ALL_TABLES	Tables accessibles
DBA_TABLES	Toutes les tables (DBA seulement)
USER_TAB_COLUMNS	Colonnes de mes tables
ALL_TAB_COLUMNS	Colonnes de toutes les tables accessibles
USER_CONSTRAINTS	Mes contraintes
USER_INDEXES	Mes index
USER_OBJECTS	Tous mes objets (tables, vues, triggers...)
DBA_SYS_PRIVS	Privilèges système
DBA_TAB_PRIVS	Privilèges objet
DBA_ROLES	Tous les rôles
DBA_ROLE_PRIVS	Rôles accordés aux utilisateurs
USER_SEGMENTS	Taille des segments (tables, index)
DBA_TABLESPACES	Informations sur les tablespaces
V$TABLESPACE	Tablespaces depuis le fichier de contrôle

Listing 18 – Requêtes utiles sur le dictionnaire

```sql
-- Trouver l'utilisateur courant
SELECT USER FROM DUAL;

-- Lister mes tables
SELECT TABLE_NAME FROM USER_TABLES;

```

```sql
-- Colonnes d'une table
SELECT COLUMN_NAME, DATA_TYPE, NULLABLE
FROM USER_TAB_COLUMNS WHERE TABLE_NAME = 'EMPLOYE';

-- Lister mes contraintes
SELECT CONSTRAINT_NAME, CONSTRAINT_TYPE, TABLE_NAME, STATUS
FROM USER_CONSTRAINTS WHERE TABLE_NAME = 'EMPLOYE';

-- Taille d'une table en Ko
SELECT SEGMENT_NAME, BYTES/1024 AS "Taille (Ko)"
FROM USER_SEGMENTS WHERE SEGMENT_NAME = 'INTERVENTIONS';

-- Voir les privil ges d'un utilisateur
SELECT * FROM DBA_SYS_PRIVS WHERE GRANTEE = 'NOM_USER';
SELECT * FROM DBA_TAB_PRIVS WHERE GRANTEE = 'NOM_USER';
```

4 Chapitre 3 – Contraintes d'intégrité et PL/SQL

4.1 Types de contraintes

- **Statiques :** portent sur un état de la BD (une valeur, une clé, une FK...).
- **Dynamiques :** portent sur un changement d'état (triggers).
- **Intra-relation :** une seule table.
- **Inter-relation :** plusieurs tables (FK, ASSERT avec sous-requête).

Listing 19 – ASSERT (non-standard, syntaxe Oracle-like)

```
-- Contrainte: un employ  ne peut pas gagner plus du double de la
   moyenne
CREATE ASSERTION ci_salaire CHECK (
  NOT EXISTS (
    SELECT * FROM Employe E
    WHERE Salaire > 2 * (SELECT AVG(Salaire) FROM Employe
                         WHERE Rayon = E.Rayon)
  )
);
```

4.2 PL/SQL – Tout ce qu'il faut savoir

4.2.1 Structure d'un bloc PL/SQL

```
DECLARE
  -- D claration des variables
  v_nom         VARCHAR2(30);
  v_salaire     NUMBER(8,2);
  v_note        NUMBER := 0;        -- initialisation
  v_ok          BOOLEAN := TRUE;
  v_emp         Employe%ROWTYPE;    -- toute une ligne de la table
  v_sal_type    Employe.Salaire%TYPE; -- type d'une colonne
BEGIN
  -- Instructions SQL et PL
  SELECT Nom, Salaire INTO v_nom, v_salaire
  FROM Employe WHERE NSS = 1001;

  DBMS_OUTPUT.PUT_LINE('Nom : ' || v_nom);
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Aucun employe trouve');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);
END;
/
```

4.2.2 Instructions de contrôle

```
-- IF / ELSIF / ELSE
IF v_note >= 10 THEN
  DBMS_OUTPUT.PUT_LINE('Recu');
ELSIF v_note >= 8 THEN
  DBMS_OUTPUT.PUT_LINE('Rattrapage');
ELSE
  DBMS_OUTPUT.PUT_LINE('Refuse');
END IF;

-- CASE (sur une variable)
CASE v_grade
  WHEN 'TS'        THEN v_coeff := 1.0;
  WHEN 'Ingenieur' THEN v_coeff := 1.5;
  ELSE                  v_coeff := 1.2;
END CASE;

-- Boucle FOR
FOR i IN 1..10 LOOP
  DBMS_OUTPUT.PUT_LINE('i = ' || i);
END LOOP;

-- Boucle WHILE
WHILE v_compteur < 100 LOOP
  v_compteur := v_compteur + 1;
END LOOP;

-- Boucle LOOP avec EXIT WHEN (= r p ter ... jusqu'  )
LOOP
  FETCH cur INTO v_nom;
  EXIT WHEN cur%NOTFOUND;
END LOOP;
```

4.2.3 SELECT INTO (mono-ligne)

```
-- R cup rer une seule ligne
SELECT Nom, Salaire INTO v_nom, v_sal
FROM Employe WHERE NSS = 1001;

-- R cup rer toute une ligne
SELECT * INTO v_emp FROM Employe WHERE NSS = 1001;
-- Acc s : v_emp.Nom, v_emp.Salaire...
```

4.2.4 Curseurs (sélection multi-lignes)

Listing 20 – Curseur explicite – patron standard

```
DECLARE
  CURSOR c1 IS
```

```
    SELECT Nom, Moyenne FROM Etudiant WHERE Specialite = 'SSI';

  v_nom      Etudiant.Nom%TYPE;
  v_moy      Etudiant.Moyenne%TYPE;
BEGIN
  OPEN c1;
  FETCH c1 INTO v_nom, v_moy;

  WHILE c1%FOUND LOOP   -- %FOUND = vrai si FETCH a ramen  une ligne
    IF v_moy >= 10 THEN
      INSERT INTO Liste_Recus VALUES (v_nom);
    ELSE
      INSERT INTO Liste_Refuses VALUES (v_nom);
    END IF;
    FETCH c1 INTO v_nom, v_moy;
  END LOOP;

  CLOSE c1;
  COMMIT;
END;
/
```

Variables système des curseurs :

- `curseur%FOUND` : VRAI si le dernier FETCH a ramené une ligne.
- `curseur%NOTFOUND` : contraire.
- `curseur%ROWCOUNT` : nombre de lignes déjà ramenées.
- `curseur%ISOPEN` : VRAI si le curseur est ouvert.

4.2.5 Procédures et fonctions stockées

Listing 21 – Procédure stockée

```
CREATE OR REPLACE PROCEDURE inscription (
  p_id      VARCHAR2,
  p_nom     VARCHAR2,
  p_spec    VARCHAR2,
  p_annee   NUMBER
)
AS
BEGIN
  INSERT INTO Etudiant VALUES (p_id, p_nom, p_spec);
  INSERT INTO Inscrire  VALUES (p_id, p_annee);
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Inscription de ' || p_nom || ' reussie');
END;
/

-- Appel depuis SQL*Plus
EXECUTE inscription('E001', 'Ali', 'SSI', 2025);
```

Listing 22 – Fonction stockée

```
CREATE OR REPLACE FUNCTION moy_joueur (p_equipe VARCHAR2)
RETURN NUMBER
IS
  v_moy NUMBER(6,2);
BEGIN
  SELECT AVG(Points) INTO v_moy
  FROM Joueur WHERE CodeEquipe = p_equipe;
  RETURN v_moy;
END;
/

-- Appel dans une requ te SQL
SELECT NomJoueur FROM Joueur
WHERE Points > moy_joueur('E1');
```

4.2.6 Gestion des exceptions

```
DECLARE
  -- D claration d'une exception utilisateur avec n   Oracle
  contrainte_violation EXCEPTION;
  PRAGMA EXCEPTION_INIT(contrainte_violation, -2291); -- FK inexistante

BEGIN
  INSERT INTO Fils VALUES (...);

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Aucune donnee');
  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Doublon interdit');
  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('Division par zero');
  WHEN contrainte_violation THEN
    DBMS_OUTPUT.PUT_LINE('Parent inexistant');
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);
END;
/
```

5 Chapitre 4 – Optimisation des requêtes

5.1 Les deux approches

— **RBO (Rule-Based Optimization) :** applique des règles algébriques sans tenir compte des statistiques. Obsolète mais encore demandé aux examens pour les arbres algébriques.

— **CBO (Cost-Based Optimization) :** utilise les statistiques (taille des tables, NDIST, min/max...) pour choisir le plan le moins cher en E/S.

5.2 Propriétés algébriques (pour construire l'arbre optimisé)

1. $\sigma_{F_1}(\sigma_{F_2}(E)) = \sigma_{F_1 \wedge F_2}(E)$ (grouper les sélections)
2. $\sigma_F(E_1 \times E_2) = \sigma_F(E_1) \times E_2$ si F porte sur E_1 (descendre la sélection)
3. $\sigma_{F_1 \wedge F_2}(E_1 \times E_2) = \sigma_{F_1}(E_1) \times \sigma_{F_2}(E_2)$ (distribuer)
4. Remplacer $\sigma_F(E_1 \times E_2)$ par $E_1 \bowtie_F E_2$ quand F compare attributs des deux tables.
5. Commutativité : $R \bowtie S = S \bowtie R$
6. Associativité : $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$

Algorithme d'optimisation (à appliquer à l'examen) :

1. Séparer les conjonctions de sélections en sélections simples.
2. Descendre chaque sélection le plus bas possible (vers les feuilles).
3. Descendre les projections le plus bas possible.
4. Grouper et combiner les opérations unaires.

5.3 Algorithmes de jointure

1. Boucles imbriquées (Nested Loop Join) :

$$\text{Coût} = |R| + |R| \times |S|$$

Si R tient en mémoire : Coût $= |R| + |S|$

2. Tri-fusion (Sort-Merge Join) :

$$\text{Coût} = 2|T_1| \log_b |T_1| + 2|T_2| \log_b |T_2| + |T_1| + |T_2|$$

(si aucune table n'est triée)

3. Hachage (Hash Join) :

$$\text{Coût} = 3(|R| + |S|)$$

4. Avec index sur S :

$$\text{Coût} = |\text{index}| + |R| + \frac{||R|| \times |S|}{\text{NDIST}(S, b)}$$

5.4 Hints Oracle

```sql
-- Forcer l'utilisation d'un index
SELECT /*+ INDEX(emp idx_grade) */ * FROM Employe emp WHERE Grade = 'TS';

-- Forcer un scan complet
SELECT /*+ FULL(emp) */ * FROM Employe emp WHERE Salaire > 20000;

-- Forcer l'ordre de jointure
SELECT /*+ ORDERED */ * FROM T1, T2, T3 WHERE ...;

-- Choisir le mode d'optimisation
ALTER SESSION SET OPTIMIZER_GOAL = ALL_ROWS;   -- minimise temps total
ALTER SESSION SET OPTIMIZER_GOAL = FIRST_ROWS; -- minimise temps pour 1re ligne
ALTER SESSION SET OPTIMIZER_GOAL = RULE;       -- RBO
```

6 SQL de A à Z – Tout ce qu'il faut maîtriser

6.1 DDL – Définition des données

Listing 23 – CREATE TABLE complet

```sql
CREATE TABLE Employe (
  NSS           NUMBER(8)     PRIMARY KEY,
  Nom           VARCHAR2(40) NOT NULL,
  Prenom        VARCHAR2(40),
  DateRecruit   DATE,
  Salaire       NUMBER(10,2) DEFAULT 0,
  Categorie     VARCHAR2(20) CONSTRAINT chk_cat CHECK (Categorie IN ('Mecanicien','Assistant')),
  NumDept       NUMBER        REFERENCES Departement(NumDept) ON DELETE SET NULL
);
```

Listing 24 – ALTER TABLE – tout savoir

```sql
-- Ajouter une colonne
ALTER TABLE Employe ADD DateInstallation DATE;

-- Modifier le type/taille d'une colonne
ALTER TABLE Employe MODIFY Prenom VARCHAR2(60);

-- Supprimer une colonne
ALTER TABLE Employe DROP COLUMN DateInstallation;

-- Renommer une colonne
ALTER TABLE Client RENAME COLUMN Adresse TO AdresseClient;

-- Ajouter une contrainte
ALTER TABLE Employe ADD CONSTRAINT chk_salaire CHECK (Salaire > 0);

-- D sactiver une contrainte (pour des imports massifs par exemple)
ALTER TABLE Employe DISABLE CONSTRAINT chk_salaire;

-- Activer une contrainte
ALTER TABLE Employe ENABLE CONSTRAINT chk_salaire;

-- Supprimer une contrainte
ALTER TABLE Employe DROP CONSTRAINT chk_salaire;
```

6.2 DML – Manipulation des données

Listing 25 – INSERT, UPDATE, DELETE

```sql
-- INSERT valeurs directes
INSERT INTO Employe (NSS, Nom, Salaire) VALUES (1001, 'Ahmed', 30000);

```

```sql
-- INSERT depuis une autre table
INSERT INTO Archive_Employe SELECT * FROM Employe WHERE Salaire <
    10000;

-- UPDATE
UPDATE Employe SET Salaire = Salaire + 5000 WHERE NSS = 64;

-- UPDATE multi-colonnes
UPDATE Employe SET Salaire = 25000, Categorie = 'Mecanicien' WHERE NSS
    = 54;

-- DELETE
DELETE FROM Vehicule WHERE NUMMODELE = 3; -- Modele "Serie 5"
```

Attention / Piège fréquent

Pour modifier sans bloquer sur une contrainte : `ALTER TABLE ... DISABLE CONSTRAINT nomC;` puis faire la modification, puis `ENABLE CONSTRAINT nomC;`.

6.3 DQL – Interrogation

Listing 26 – SELECT – syntaxe générale

```sql
SELECT [DISTINCT] col1, col2, fonction(col3) AS alias
FROM table1 t1, table2 t2            -- ou JOIN
WHERE condition_jointure
  AND condition_selection
GROUP BY col1, col2
HAVING condition_sur_groupe
ORDER BY col1 [ASC|DESC], col2;
```

Listing 27 – Requêtes sur la BD du TP (exemples des TPs 1-3)

```sql
-- Lister les mod les et leur marque
SELECT mo.MODELE, ma.MARQUE
FROM MODELE mo, MARQUE ma
WHERE mo.NUMMARQUE = ma.NUMMARQUE;

-- V hicules ayant au moins une intervention
SELECT DISTINCT v.NUMVEHICULE, v.NUMIMMAT
FROM VEHICULE v
WHERE v.NUMVEHICULE IN (SELECT NUMVEHICULE FROM INTERVENTIONS);

-- Dur e moyenne d'une intervention (en jours)
SELECT AVG(DATEFININTERV - DATEDEBINTERV) AS "Duree moyenne (jours)"
FROM INTERVENTIONS;

-- Montant global des interventions co tant > 30000 DA
SELECT SUM(COUTINTERV) AS MontantGlobal
FROM INTERVENTIONS
WHERE COUTINTERV > 30000;
```

```sql

-- Employé(s) avec le plus grand nombre d'interventions
SELECT NUMEMPLOYE, COUNT(*) AS NbInterv
FROM INTERVENANTS
GROUP BY NUMEMPLOYE
HAVING COUNT(*) = (SELECT MAX(COUNT(*)) FROM INTERVENANTS GROUP BY
    NUMEMPLOYE);
```

6.4 Fonctions SQL utiles

Fonction	Utilisation	Exemple
COUNT(*)	Nombre de lignes	COUNT(*) AS n
SUM(col)	Somme	SUM(Salaire)
AVG(col)	Moyenne	AVG(Coutinterv)
MAX(col)	Maximum	MAX(Note)
MIN(col)	Minimum	MIN(Salaire)
TO_DATE(str, fmt)	Conversion texte → date	TO_DATE('01-01-2024','DD-MM-YYYY')
TO_CHAR(val)	Conversion en texte	TO_CHAR(Salaire)
NVL(col, val)	Remplace NULL	NVL(Salaire, 0)
SYSDATE	Date du jour	WHERE Date > SYSDATE
INTERVAL '1' YEAR	Intervalle temporel	Date + INTERVAL '1' YEAR

6.5 Sous-requêtes et opérateurs ensemblistes

```sql
-- Sous-requête corrélée
SELECT Nom FROM Employe E
WHERE Salaire > (SELECT AVG(Salaire) FROM Employe E2
                 WHERE E2.NumDept = E.NumDept);

-- EXISTS
SELECT Nom FROM Employe E
WHERE EXISTS (SELECT 1 FROM Intervenants I WHERE I.NumEmploye = E.
    NumEmploye);

-- IN
SELECT * FROM Vehicule WHERE NUMMODELE IN (SELECT NUMMODELE FROM MODELE
     WHERE NUMMARQUE = 7);

-- Opérateurs ensemblistes
SELECT NSS FROM Employe UNION SELECT NumEmp FROM Archive;
SELECT NSS FROM Employe MINUS SELECT NumEmp FROM Archive;
SELECT NSS FROM Employe INTERSECT SELECT NumEmp FROM Archive;
```

7 Résumé des TPs

7.1 TP1 – Création et manipulation de BD

BD utilisée : Gestion des interventions mécaniques – tables CLIENT, EMPLOYE, MARQUE, MODELE, VEHICULE, INTERVENTIONS, INTERVENANTS.

Listing 28 – TP1 – Travail demandé (réponses clés)

```sql
-- Partie I : Tablespaces et utilisateur
CREATE TABLESPACE INTERVENTION_TBS
  DATAFILE 'C:\intervention_tbs.dat' SIZE 100M AUTOEXTEND ON ONLINE;

CREATE TEMPORARY TABLESPACE INTERVENTION_TempTBS
  TEMPFILE 'C:\intervention_temp.dat' SIZE 100M AUTOEXTEND ON;

CREATE USER DBAINTERVENTION IDENTIFIED BY pass123
  DEFAULT TABLESPACE INTERVENTION_TBS
  TEMPORARY TABLESPACE INTERVENTION_TempTBS;

GRANT ALL PRIVILEGES TO DBAINTERVENTION;

-- Partie II : Contrainte dynamique (Q10)
ALTER TABLE INTERVENTIONS ADD CONSTRAINT chk_dates_interv
  CHECK (DATEFININTERV > DATEDEBINTERV);
-- OU mieux : trigger pour les mises    jour aussi

-- Partie III -- Q12 : Augmenter salaire de BADI Hatem
UPDATE EMPLOYE SET SALAIRE = SALAIRE + 5000
WHERE NOMEMP = 'BADI' AND PRENOMEMP = 'Hatem';

-- Partie III -- Q13 : +5 jours aux interventions de F vrier
ALTER TABLE INTERVENTIONS DISABLE CONSTRAINT chk_dates_interv;
UPDATE INTERVENTIONS SET DATEDEBINTERV = DATEDEBINTERV + 5
WHERE TO_CHAR(DATEDEBINTERV, 'MM') = '02';
ALTER TABLE INTERVENTIONS ENABLE CONSTRAINT chk_dates_interv;

-- Partie III -- Q14 : Supprimer v hicules de mod le S rie 5
-- NUMMODELE = 3 correspond    S rie 5
-- PROBL ME : des v hicules ont des interventions (FK)
-- SOLUTION 1 : ON DELETE CASCADE sur INTERVENTIONS.NUMVEHICULE*
-- SOLUTION 2 : Supprimer d'abord les interventions
DELETE FROM INTERVENANTS WHERE NUMINTERVENTION IN
  (SELECT NUMINTERVENTION FROM INTERVENTIONS WHERE NUMVEHICULE IN
    (SELECT NUMVEHICULE FROM VEHICULE WHERE NUMMODELE = 3));
DELETE FROM INTERVENTIONS WHERE NUMVEHICULE IN
  (SELECT NUMVEHICULE FROM VEHICULE WHERE NUMMODELE = 3);
DELETE FROM VEHICULE WHERE NUMMODELE = 3;
```

7.2 TP2 – Gestion des droits

Listing 29 – TP2 – Réponses clés

```
-- Q1 : Cr er l'utilisateur Admin depuis DBAINTERVENTION
CREATE USER Admin IDENTIFIED BY admin123
  DEFAULT TABLESPACE INTERVENTION_TBS
  TEMPORARY TABLESPACE INTERVENTION_TempTBS;

-- Q3 : Donner CREATE SESSION
GRANT CREATE SESSION TO Admin;

-- Q4 : Donner privil ges de cr ation
GRANT CREATE TABLE, CREATE USER TO Admin;

-- Q6 : Donner SELECT sur EMPLOYE    Admin
GRANT SELECT ON DBAINTERVENTION.EMPLOYE TO Admin;

-- Q7 : Augmenter salaires des employ s avec > 5 interventions
-- Admin doit avoir UPDATE sur EMPLOYE + SELECT sur INTERVENANTS
UPDATE DBAINTERVENTION.EMPLOYE
SET SALAIRE = SALAIRE + 5000
WHERE NUMEMPLOYE IN (
  SELECT NUMEMPLOYE FROM DBAINTERVENTION.INTERVENANTS
  GROUP BY NUMEMPLOYE HAVING COUNT(*) > 5
);

-- Q8 : Donner les droits n cessaires
GRANT UPDATE ON DBAINTERVENTION.EMPLOYE TO Admin;
GRANT SELECT ON DBAINTERVENTION.INTERVENANTS TO Admin;

-- Q9 : Cr er un index
CREATE INDEX NOMEMP_IX ON DBAINTERVENTION.EMPLOYE(NOMEMP);
-- Admin doit avoir le privil ge INDEX sur la table
GRANT INDEX ON DBAINTERVENTION.EMPLOYE TO Admin;
```

7.3 TP3 – Dictionnaire de données

Listing 30 – TP3 – Requêtes clés sur le dictionnaire

```
-- Q1 : Structure du catalogue DICT
DESCRIBE DICT;
SELECT * FROM DICT;            -- contient toutes les vues du
   dictionnaire

-- Q3 : Trouver l'utilisateur courant SANS show user
SELECT USER FROM DUAL;         -- ou : SELECT USERNAME FROM USER_USERS;

-- Q5 : V rifier que les tables du TP1 existent
SELECT TABLE_NAME FROM USER_TABLES;

-- Q7 : Description des colonnes de VEHICULE
SELECT COLUMN_NAME, DATA_TYPE, DATA_LENGTH, NULLABLE
FROM USER_TAB_COLUMNS WHERE TABLE_NAME = 'VEHICULE';
```

```sql

-- Q8 : V rifier FK entre VEHICULE et INTERVENTIONS
SELECT uc.CONSTRAINT_NAME, uc.CONSTRAINT_TYPE,
       ucc.COLUMN_NAME, uc.R_CONSTRAINT_NAME
FROM USER_CONSTRAINTS uc
JOIN USER_CONS_COLUMNS ucc ON uc.CONSTRAINT_NAME = ucc.CONSTRAINT_NAME
WHERE uc.TABLE_NAME IN ('VEHICULE','INTERVENTIONS')
  AND uc.CONSTRAINT_TYPE = 'R';

-- Q9 : Toutes les contraintes du TP1
SELECT TABLE_NAME, CONSTRAINT_NAME, CONSTRAINT_TYPE, STATUS
FROM USER_CONSTRAINTS
WHERE TABLE_NAME IN ('CLIENT','EMPLOYE','MARQUE','MODELE','VEHICULE',
                     'INTERVENTIONS','INTERVENANTS')
ORDER BY TABLE_NAME, CONSTRAINT_TYPE;

-- Q11 : Privil ges accord s    Admin (depuis System)
SELECT * FROM DBA_SYS_PRIVS WHERE GRANTEE = 'ADMIN';
SELECT * FROM DBA_TAB_PRIVS WHERE GRANTEE = 'ADMIN';

-- Q14 : Trouver le propri taire d'une table
SELECT OWNER FROM ALL_TABLES WHERE TABLE_NAME = 'INTERVENTIONS';

-- Q15 : Taille en Ko
SELECT SEGMENT_NAME, BYTES/1024 AS "Ko"
FROM USER_SEGMENTS WHERE SEGMENT_NAME = 'INTERVENTIONS';
```

8 Index B-arbre (question 8 des examens 2017 et docx)

Question répétée aux examens

Question répétée exactement identique en 2017 et dans le sujet docx : Dessiner le B-arbre d'ordre 5 pour la séquence : $\{1, 50, 7, 153, 44, 8, 77, 6, 10, 105, 9, 13, 38, 2, 89, 19, 5, 18, 37, 49, 23, 15, 100, 28, 3, 78, 4, 60, 46\}$

Règles d'un B-arbre d'ordre p :

- Chaque nœud interne a au maximum p enfants et $p - 1$ clés.
- Chaque nœud interne (sauf la racine) a au minimum $\lceil p/2 \rceil$ enfants.
- Les feuilles peuvent contenir au maximum $\lceil p/2 \rceil - 1$ à $p - 1$ clés (selon la variante).
- Dans cet exam : feuilles contiennent **au maximum 5 enregistrements**, ordre 5.

Principe d'insertion :

1. Insérer dans la feuille appropriée.
2. Si la feuille est pleine (5 clés), la **scinder** (split) : la clé médiane monte dans le nœud parent.
3. Si le nœud parent est plein, le scinder aussi (propagation).

Exemple de début d'insertion (ordre 5, max 5 clés/feuille) :

- Après $\{1, 50, 7, 153, 44\}$: une feuille avec ces 5 valeurs triées : $[1, 7, 44, 50, 153]$.
- Insertion de 8 : feuille pleine $\rightarrow$ split. Médiane = 44 monte. Résultat : racine $[44]$, feuille gauche $[1, 7, 8]$, feuille droite $[50, 153]$.

(La construction complète est longue à dessiner. L'essentiel est de connaître les règles et de montrer 2-3 états intermédiaires.)

Comparaison index B-arbre vs Bitmap :

	B-arbre	**Bitmap**
Cardinalité attribut	Haute (NSS, dates...)	Faible (2-10 valeurs distinctes)
Requêtes	Égalité, intervalle	Égalité, combinaisons AND/OR
Mise à jour	Efficace	Coûteuse sur tables modifiées fréquemment
Taille	$O(n \log n)$	$\|\|T\|\| \times$ NDIST/8 octets
Usage	OLTP	Data warehouse, reporting

9 SQL*Loader (chargement de données depuis fichier)

Question répétée aux examens

Question répétée dans 2017 et docx (exactement le même fichier .ctl) : Expliquer le fichier de contrôle sinistres.ctl et donner la commande de chargement.

Listing 31 – Fichier de contrôle .ctl (syntaxe)

```
LOAD DATA
INFILE 'F:\sinistres.csv'          -- fichier source des donn es
INTO TABLE SINISTRE                 -- table cible
TRAILING NULLCOLS                   -- colonnes absentes = NULL
(
  NOSINISTRE POSITION(1:10)  DECIMAL,
  NOCONTRAT  POSITION(11:20) DECIMAL,
  DATE_S     POSITION(21:30) DATE "DD/MM/YYYY",
  CAUSE      POSITION(31:100) CHAR(70)
)
```

Listing 32 – Commande SQL*Loader

```
sqlldr userid=user/password control=sinistres.ctl log=sinistres.log
   direct=true
```

Signification de chaque mot-clé :

- `LOAD DATA` : indique que c'est un fichier de contrôle de chargement.
- `INFILE` : chemin vers le fichier de données source.
- `INTO TABLE` : table Oracle cible.
- `TRAILING NULLCOLS` : si une colonne est absente à la fin d'une ligne, mettre NULL.
- `POSITION(x:y)` : lire les caractères de la position x à y dans la ligne.
- `direct=true` : chargement direct (bypasse le buffer cache), beaucoup plus rapide.
- `log=` : fichier journal pour les erreurs.

10 Fiche de révision ultra-rapide (dernière heure avant l'exam)

9 choses à maîtriser pour passer l'exam

1. **CREATE TABLE avec toutes les contraintes** (PK, FK, CHECK, NOT NULL, UNIQUE)
2. **ALTER TABLE ADD/MODIFY/DROP CONSTRAINT** – ajouter des contraintes après coup
3. **Trigger BEFORE INSERT** – vérifier une contrainte dynamique avec RAISE_APPLICATION_ERROR
4. **Trigger AFTER INSERT** – mettre à jour un agrégat automatiquement
5. **CREATE USER + TABLESPACE + ROLE + PROFILE + GRANT**
6. **Cas d'échec d'un INSERT** : PK dupliquée, FK inexistante, violation trigger, droit manquant, tablespace plein
7. **Lire un plan d'exécution** : TABLE ACCESS FULL, INDEX UNIQUE SCAN, HASH JOIN, NESTED LOOPS
8. **Arbre algébrique + optimisation RBO** : descendre les σ et π, calculer le coût CBO
9. **Fragmentation PARTITION BY RANGE / LIST / REFERENCE**

Questions répétées mot pour mot

— **2017 = docx** : Même BD assurance, mêmes questions 1 à 9. La table CONTRAT, le trigger de date sinistre, les cas d'échec du même INSERT, le même plan d'exécution, le même B-arbre avec les mêmes valeurs.

— **Tous les examens** : Créer table avec contraintes + trigger contrainte dynamique + user/tablespace/rôle.

Syntaxes PL/SQL à mémoriser par cœur

```
-- Template trigger universel
CREATE OR REPLACE TRIGGER nom
BEFORE|AFTER INSERT|UPDATE|DELETE ON table
FOR EACH ROW
DECLARE variable type;
BEGIN
  SELECT col INTO var FROM T WHERE id = :NEW.id;
  IF condition THEN
    RAISE_APPLICATION_ERROR(-20001, 'Message');
  END IF;
END;
/

-- Template curseur universel
DECLARE
  CURSOR c IS SELECT col FROM T WHERE cond;
  v T.col%TYPE;
BEGIN
  OPEN c; FETCH c INTO v;
  WHILE c%FOUND LOOP
    -- traitement
    FETCH c INTO v;
  END LOOP;
  CLOSE c;
END;
/
```